

ELTE IK - Computer Science BSc

Final Exam Topics

19. Concurrent programming

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

TODO: Only the “C” is worked out, the “2018” one is not!

Distributed systems and concurrent programming

A, C: Concept of a process, properties and structure of distributed systems, naming systems, communication, synchronization, consistency.

B: Specification of tasks with safety and progress conditions, properties of an abstract parallel program, concept of a solution, solving notable tasks with parallel and distributed programs.

2018: Thread handling. Scheduling, context switch. Race condition. Synchronization. Blocking operations. Use of memory (stack and heap) in threads. The language tools of concurrent programming. Data structures usable for synchronization and communication.

1 Processes, threads

Thread: The thread is a kind of software counterpart of the processor, with minimal context. If we stop the thread, the context can be saved and reloaded for continued execution.

Process: The process (process or task) is a larger unit gathering one or more threads. The threads of a process work on a common memory area (address space), but different processes do not see each other's memory area.

Context switch: The handing of control to another process/thread, so that one processor can execute several threads/processes.

Thread vs. process: For switching between threads the services of the OS do not have to be used, while for switching between processes, so that the memory area of the old and new process is separated, a good part of the content of the memory controller (MMU) has to be rewritten, for which only the kernel level has the right. The creation, deletion of processes and the context switch between them is much more costly than for threads.

2 Properties and structure of distributed systems

Concept of a distributed system: A distributed system is a collection of independent computers that, to its users, appears as a single coherent system.

2.1 The goals, properties of a distributed system

The goals of a distributed system are the following:

- Making remote resources accessible
- Transparency
- Openness
- Scalability

2.1.1 Transparency

Transparency is nothing other than the hiding of various information related to the resources from the user. Based on what we hide, there are several kinds:

Fajta	Angolul	Mit rejt el az erőforrással kapcsolatban?
Hozzáférési/elérési	Access	Adatábrázolás; elérés technikai részletei
Elhelyezési	Location	Fizikai elhelyezkedés
Áthelyezési	Migration	Elhelyezési + a hely meg is változhat
Mozgatási	Relocation	Áthelyezési + használat közben is történhet az áthelyezés
Többszörözési	Replication	Az erőforrásnak több másolata is lehet a rendszerben
Egyidejűségi	Concurrency	Több versenyhelyzetű felhasználó is elérheti egyszerre
Meghibásodási	Failure	Meghibásodhat és újra üzembe állhat

Figure 1: The different types of transparency.

2.1.2 Openness

The system is able to provide services for other open systems, and to use their services:

- The systems have well-defined interfaces.
- They support the portability of applications as much as possible.
- The cooperation of systems is easily achievable.

An open distributed system should be easily applicable in a heterogeneous environment, that is, on different hardware, platforms, programming languages.

Its implementation:

- It is important that the system consist of easily replaceable elements.
- Use of internal interfaces, not a single monolithic system.
- The system has to be as parameterizable as possible.
- The change/replacement of a single component should affect the other parts of the system as little as possible.

2.1.3 Scalability

It has several meanings, 3 important dimensions:

1. scalability by size: the number of users and/or processes
2. geographical scalability: the largest distance between the vertices
3. administrative scalability: the number of administrative domains

Of these, most systems handle scalability by size; one possible way of realizing this is the use of stronger servers. The other two are harder to handle.

Techniques for realizing scalability:

- Hiding the communication latency by performing another activity while waiting for the response. For this asynchronous communication is needed.
- Distribution: the data and computations are stored/performed by several computers (e.g. what is possible, we have the client compute, use of distributed naming systems, etc.)
- Replication/caching: Several computers store copies of a piece of data

Scalability has a price. Maintaining several copies can lead to inconsistency (if we modify one, it can differ from the others). This can be avoided with global synchronization (after every single change we update all copies), but global synchronization scales badly. Because of this, in many cases we have to give up global synchronization, which, however, results in a certain degree of inconsistency. It is system-dependent to what degree this is allowed. The goal is that the degree of inconsistency stay below the allowed level.

2.2 Types of distributed systems

Main types:

- Distributed computing systems:
- Distributed information systems
- Distributed pervasive systems

2.2.1 Distributed computing systems

Its goal is to perform computations with high performance.

Cluster: A collection of computers connected to a local network. A homogeneous system (the same OS, hardware-wise similar), with centralized control (generally on one machine).

Grid Can extend even to large networks, can span even several organizational units. It is characterized by a heterogeneous architecture.

Cloud: A multi-layer architecture: hardware, infrastructure, platform, application.

2.2.2 Distributed information systems

The primary goal is generally the handling of data, or the access of other information systems. For example transaction-handling systems.

A transaction is an operation performed on a collection of data (e.g. a database, database object, etc.) (it can have sub-operations). The following requirements are usually imposed on transactions (ACID):

- Atomic, elementary (atomicity): Either the whole transaction takes place with all its sub-operations, or the data warehouse does not change at all.
- Consistent (consistency): We say of the data store that it is valid if certain conditions formulated for the given data store hold. A transaction is consistent if it produces a valid state at the end of the transaction.
- Isolatable, serializable (isolation): Transactions happening simultaneously give a result as if they had been executed one after another.
- Durability (durability): After execution we save the result to durable storage, so it can be restored in case of a crash.

2.3 Structure of distributed systems

Basic idea: Let us organize the elements of the system, according to their logical role, into different components, and distribute these over the machines of the system.

2.3.1 Centralized architectures

Client-server model: Some processes (servers) offer services, while other processes (clients) want to use these services. The client sends a request to the server, to which the server responds, and so it uses the service. The client and server processes can be on different machines.

2.3.2 Multi-layer architectures

Distributed information systems are often divided into three logical layers (layer or tier):

- Presentation: consists of the components forming the user interface of the application.
- Business logic: describes the operation of the application without concrete data
- Persistence: the durable storage of data

2.3.3 Decentralized architectures

Peer-to-peer (P2P): Among the vertices (peers) there are mostly none with distinguished roles.

Overlay network: The vertices adjacent in the graph can physically be far from each other, the system hides that the communication between them happens through several machines. Most P2P systems are built on an overlay network.

Types of P2P systems:

- Structured P2P: The graph structure produced by the vertices is fixed. We organize the vertices according to some structure into an overlay network and from the vertices, based on their identifier, services can be used. E.g.: distributed hash table (DHT).
- Unstructured P2P: Such systems try to maintain a random graph structure. Every vertex has only a partial view of the graph. Every vertex P from time to time randomly chooses a Q neighbor. P and Q exchange information and send each other the vertices they know.
- Hybrid P2P: some vertices have a special role

Superpeer: A vertex that has a separate task, e.g. maintaining an index for searching, supervising the state of the network, creating connections between vertices.

3 Naming systems

The entities of distributed systems are reached through their access points. These are identified remotely by their address, which names the given point.

It can be practical to name the entity independently of its access points too. Such names are location-independent.

Simple name: Has no structure, contains random text. Can only be used for comparison.

Identifier: A name is an identifier if it is in a one-to-one relationship with the named entity, and this assignment is permanent, that is, the name cannot later refer to another entity.

3.1 Unstructured names

3.1.1 Simple solutions

Broadcasting: We announce the identifier on the network. The entity sends back its current address. Its disadvantages:

- It does not scale beyond local networks.
- Every machine on the network has to watch for the incoming request.

Forwarding pointer: When the entity moves, a pointer remains after it to its new place.

- It is hidden from the client that the software resolves a chain of forwarding pointers.
- The found address can be sent back to the client, so the further resolutions go faster.
- Geographical scaling problems:
 - The long chains are not fault-tolerant.
 - The resolution takes a long time.
 - A separate mechanism is needed for shortening the chains.

3.1.2 Home-based solutions

Single-layer system: A home belongs to the entity, this keeps track of the entity's current address. The home address (HA) of the entity is registered in a name service. The home keeps track of the current address (foreign address - FA). The client connects to the home, and gets the address from there.

Two-layer system: In the individual neighborhoods we record which entities reside nearby. Name resolution first examines this registry and if the entity is not in the neighborhood, then the home has to be turned to.

3.1.3 Distributed hash table

We make a distributed hash table (DHT), in this vertices store entities. The N vertices are organized into a ring overlay structure. To every vertex we assign an m -bit identifier, and to every entity an m -bit key ($N \leq 2^m$). The one responsible for the entity with key k is the vertex with identifier id for which $k \leq id$, and there is no other vertex between them. This vertex is also usually called the successor of the key: $succ(k)$. Every vertex p stores an FT_p finger table with m entries: $FT_p[i] = succ(p + 2^{i-1})$. We want to achieve a binary(-like) search, so every step halves the search range. To look up the entity with key k (if the current vertex does not contain it) we forward the request to the vertex with index j for which $FT_p[j] \leq k < FT_p[j+1]$, and if $p < k < FT_p[1]$, then we also direct the request to $FT_p[1]$.

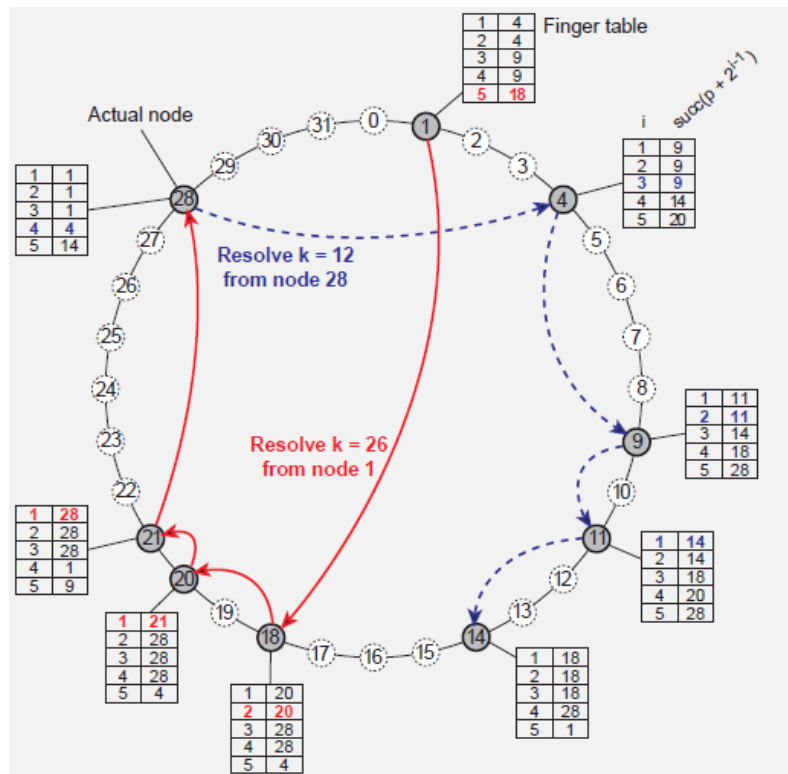


Figure 2: Example of a DHT with finger table.

3.1.4 Hierarchical methods

Hierarchical Location Services (HLS): Let us divide the network into domains, and to every domain let there belong a catalog. Let us build a hierarchy from the catalogs.

The data stored in the vertices:

- The address of entity E is found in a leaf.
- On the path leading from the root to the leaf of E , in every internal vertex there is a pointer to the next vertex down on the path.
- Since the root is the starting point of every path, it has information about every entity.

Search in the tree: The search starts from the client's domain. We go up in the tree until we reach a vertex that knows about E , then we follow the pointers to the leaf that knows E 's address. Since the root knows every entity, termination is guaranteed.

Insertion in the tree: We go up in the tree as far as for searching, then in the internal vertices we place pointers.

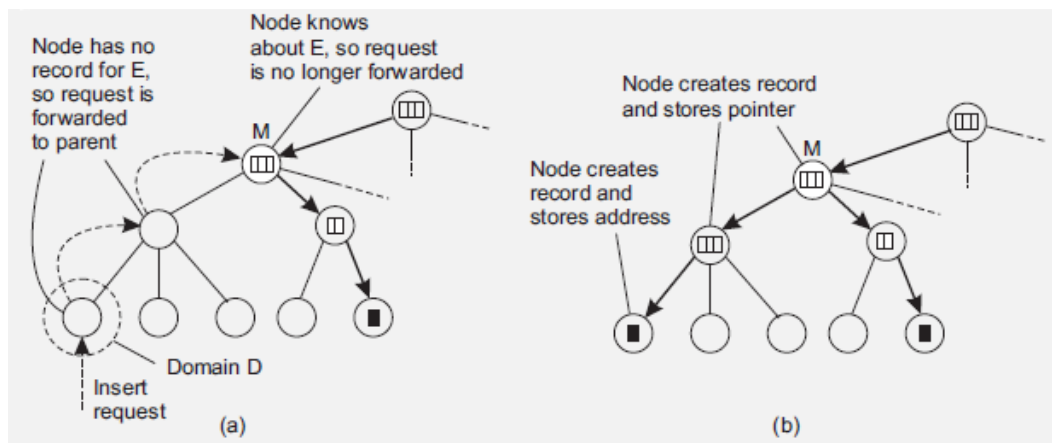


Figure 3: Insertion in the tree for HLS.

3.2 Structured names

Namespace: A rooted, directed, edge-labeled graph, the leaves contain the named entities, the internal vertices we call catalogs or directories. Reading together the labels of the path leading to the entity, we get a name of the entity. The traversed path, if it starts from the root, is an absolute path name, if it starts from an internal vertex, a relative path name. Since several paths can lead to an entity, it can have several names.

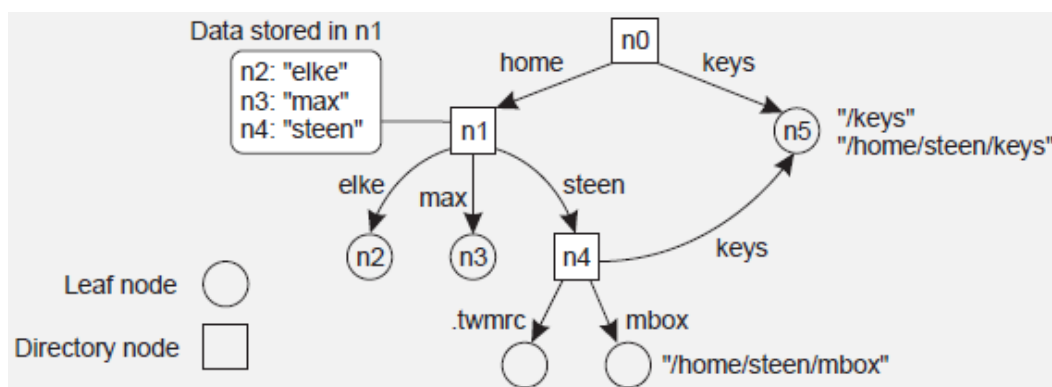


Figure 4: Example of a namespace.

In the vertices of the namespace (whether leaf or internal vertex) we can also store various attributes, e.g. the entity's type, identifier, place/address, other names, etc.

Name resolution: A starting vertex is needed to begin name resolution. The reachability of the root is ensured by the environment depending on the nature of the name, e.g.:

- www.inf.elte.hu : a DNS name server
- /home/steen/mbox : the local NFS file server
- 0031204447784 : the telephone network
- 157.181.161.79 : the path leading to the www.inf.elte.hu web server

Implementation of the namespace - DNS: If we have a large namespace, the graph has to be distributed among the machines, so that we make name resolution and the handling of the namespace efficient. Such a large namespace is the DNS (Domain Name System).

We basically distinguish 3 levels of the DNS namespace:

- Global level: The root and the upper vertices (TLDs, e.g. vertices belonging to countries - .hu, .uk, etc.) belong here. The organizations manage this jointly.

- Organizational level: The level of vertices managed by an individual organization (e.g. elte.hu, etc.).
- Managerial level: The vertices managed within a given organization (e.g. vertices within elte.hu)

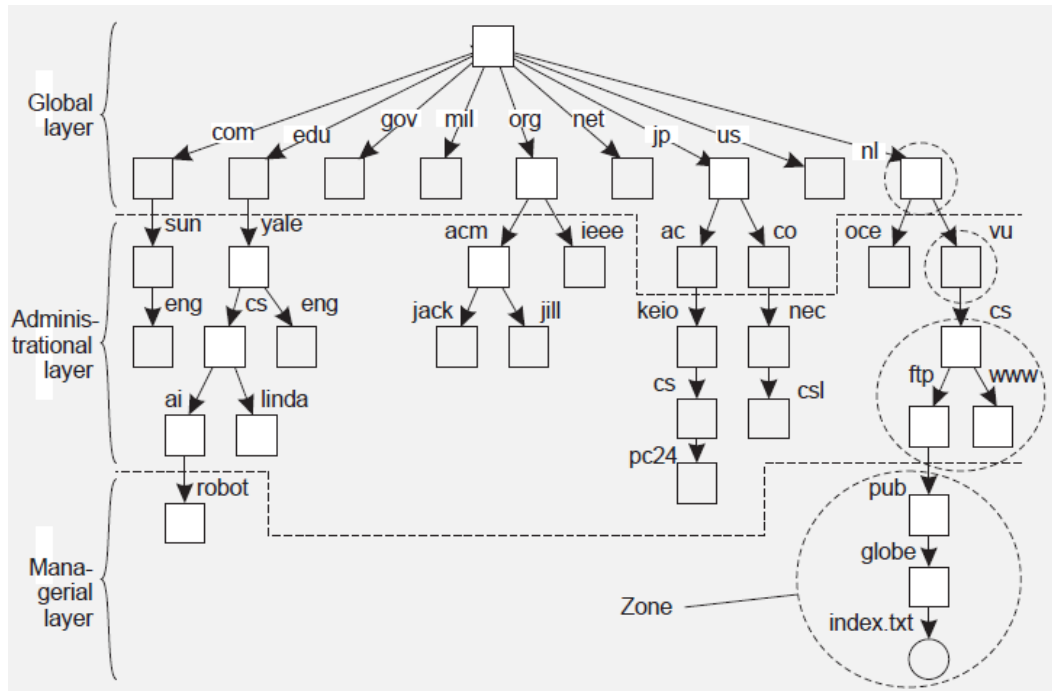


Figure 5: A part of the DNS namespace.

The different approaches to name resolution: In the case of the DNS namespace we basically apply two different name-resolution approaches:

- Recursive name resolution: During recursive name resolution the name servers, communicating among each other, resolve the names, and the response arrives immediately to the client-side resolver.
- Iterative name resolution: We start the name resolution from one of the root name servers. During iterative name resolution we always resolve only one component of the name, and the addressed name server returns the address of the name server belonging to this (if the client-side resolver receives this address, it requests the resolution of the next component from this name server - this goes on until we fully resolve the name).

Scalability: Since many requests have to be handled in a short time, the name servers of the global level would get a great load. Since at the upper levels the graph rarely changes, the data of the vertices found on these levels can be copied on several servers too, so we can start the search from closer (e.g. there are several root name servers, we turn to the one nearest to us).

3.2.1 Attribute-based names

It can often be convenient to search for entities based on their properties (attributes), but if we can give attribute values in any combination, then for the search we have to touch all entities, which is not efficient.

X.500, LDAP: In catalog services constraints are valid on the attributes (X.500 standard), which is usually accessed through the LDAP protocol. The naming system is tree-structured, its edges are addressed with attribute-value pairs. The characteristics of their path relate to the entities, and they can contain further pairs.

4 Communication

4.1 Middleware

To the middleware are usually counted such services and protocols that can be useful for many kinds of applications and basically serve as a connecting link between the entities of the system.

- Communication protocols
- Serialization (marshalling), transformation of the representation of data
- Naming protocols for facilitating the sharing of resources
- Security protocols for making the communication safer
- Scaling mechanisms for the replication and caching of data

4.2 Types of communication

Communication can be:

- transient or persistent:
 - transient: the communication system discards the message if it cannot be delivered
 - persistent: the communication system is willing to store the message for a longer time
- synchronous or asynchronous
 - synchronous: the sender waits for the response, blocks until then
 - asynchronous: the sender does not wait for the response, but continues with another activity

4.2.1 Client-server model

The client-server model typically performs transient, synchronous communication, where the client and the server have to be active simultaneously. The client, after sending the request, blocks, waiting for the server's response. The server only deals with receiving the clients and processing the requests.

4.2.2 Remote procedure call (RPC)

In a remote procedure call we want to run a subprogram on a remote machine. For this network communication is needed, which we hide with a procedure call.

The steps of the call:

1. The client process locally calls the client stub.
2. The client stub packs the identifier and parameters of the procedure. It calls the OS.
3. The OS of the local machine sends the packet to the OS of the remote machine.
4. It passes the message to the server stub.
5. The server stub unpacks the identifier and the parameters, which it passes to the server process.
6. The server process locally calls the procedure, gets the return value.
7. The sending back of the return value to the client process happens similarly, in the reverse direction.

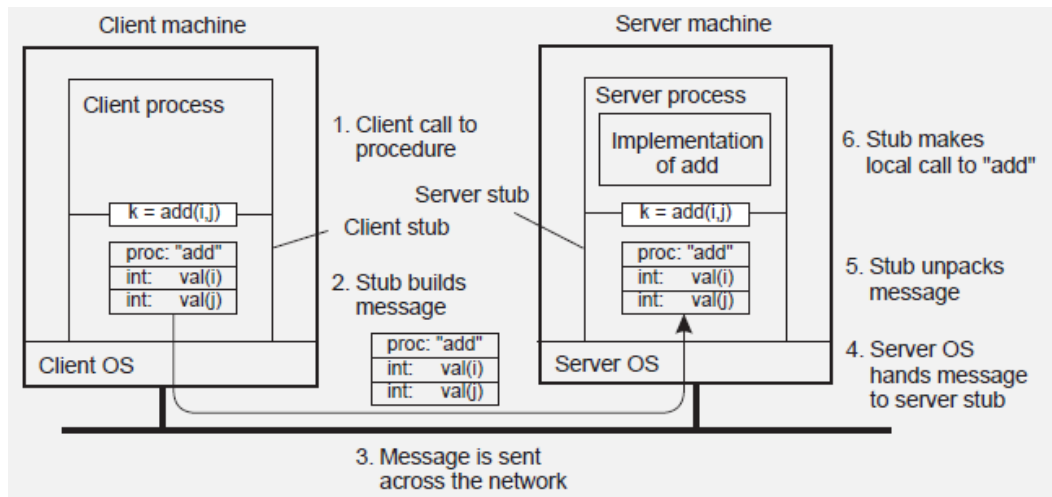


Figure 6: The steps of the remote procedure call.

4.2.3 Socket

A way of transient communication.

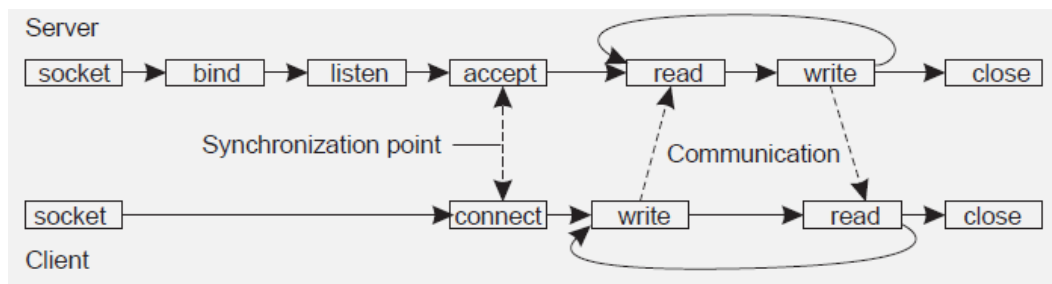


Figure 7: Communication with a socket.

4.2.4 Message-oriented middleware (MOM)

The message-oriented middleware (MOM) is a persistent, asynchronous communication architecture. With its help the processes can send messages to each other. The sending party does not have to wait for the response, until then it can deal with something else.

The MOM maintains waiting queues on the machines of the system. The clients can use the following operations on the waiting queues:

- **PUT:** Puts a message at the end of the queue.
- **GET:** Blocks until the queue is empty, then takes out the first message
- **POLL:** Queries whether there is a message. If there is, takes off the first. If there is not, it does not block, continues its activity.
- **NOTIFY:** Installs a handler routine to the waiting queue, which is called for every incoming message.

The message-queue-handling systems assume that every element of the system uses a common protocol, that is, that the structure and data representation of the messages coincide. The question: what if our system is heterogeneous? For this serves the message broker, which in a heterogeneous system takes care of the appropriate conversions, that is, transforms the message to the format used by the receiver. It generally also works as a proxy, that is, besides the mediation it also provides other functions, e.g. security functions.

4.2.5 Stream

What is common in the communication types discussed so far is that the temporal relationship between the data units does not influence their meaning, but for continuous media (e.g. audio, video, sensor data) the data is time-dependent, so regarding the temporality of the communication we make an isochronous constraint, which gives both an upper and a lower bound on the time of the transmission of the packets.

Stream: A communication form enabling such isochronous data transmission is the stream. Its main characteristics:

- Unidirectional
- Mostly directed from one source toward one or more sinks
- The source and/or sink are often directly connected to hardware elements such as a camera, screen, microphone, etc.

Its main types:

- Simple stream: transmits one kind of data, e.g. a single audio channel, or only video.
- Complex stream: Transmits several kinds of data at once, e.g. video with multi-channel audio (stereo, 5.1, etc.). In the case of a complex stream it has to be ensured that the substreams do not drift apart in time relative to each other at the sink. One way of this is synchronization. Another possible method is multiplexing and demultiplexing. Then the source makes a single stream (multiplexing). Here the substreams are guaranteed to be in sync with each other. At the sink the stream has to be broken apart into substreams (demultiplexing).

QoS: Many kinds of requirements can be prescribed in connection with streams, these by a collective name we call the quality of service (QoS). Such characteristics are, for example, the following:

- The transmission speed, that is, the bitrate.
- The largest allowed latency of starting the stream.
- The data units of the stream have to reach the sink from the source within a specified time.
- Jitter: the unevenness of the arrival time of the data units. One way of decreasing this is buffering.

5 Synchronization

5.1 Synchronization of clocks

Sometimes we want to know the exact time, sometimes it is enough if of two time points it can be determined which was earlier. The world time: UTC.

5.1.1 Physical clocks

The spreading of physical time: If there is a UTC receiver in our system, it gets the exact time. This we can spread within the system taking the following into account.

- According to the own clock of machine p , the time at the UTC moment t is $C_p(t)$
- In the ideal case the clock is always exact, that is, $C_p(t) = t$ for every UTC moment t . In other words, the speed of the clock is always 1, that is, $dC/dt = 1$.

- In reality p 's clock is either too fast or too slow, but relatively exact:

$$1 - \rho \leq \frac{dC}{dt} \leq 1 + \rho$$

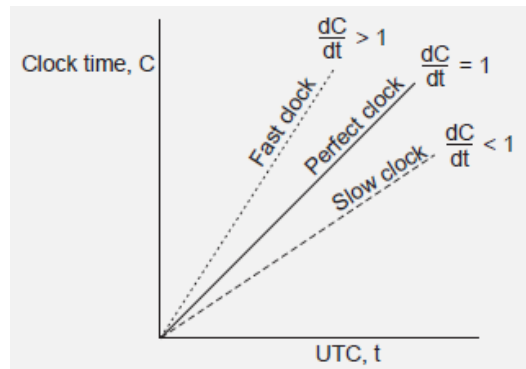


Figure 8: The speed of the clock.

Cristian's algorithm: We only want to allow a given deviation δ in the speed of the clock. Every machine queries the exact time from a central time server at most every $\frac{\delta}{2\rho}$ seconds (then we can stay within δ deviation). The clock should not be set to the received time point: it has to be taken into account that the server handled the request and the response had to arrive back over the network.

Berkeley's algorithm: The goal is not the setting of the exact time, only that the time of every machine within the system be the same. The time server from time to time requests the time of every machine, from which it takes an average, then notifies everyone how much they have to adjust their own clock by. The time may not flow backwards at any machine, so if some clock had to be set back, then instead it slows down its clock until the wanted time is reached.

5.1.2 Logical clocks

The happened-before relation: The happened-before relation is a relation with the following properties. The notation that a happened before b : $a \rightarrow b$.

- If in the same process a occurred before b , then $a \rightarrow b$.
- If event a is the sending of a message, and b is the receipt of this message, then $a \rightarrow b$.
- Transitive: If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

Time and the happened-before relation: To every event e we assign a timestamp, which is an integer. Its notation: $C(e)$, and we require the following properties:

- If $a \rightarrow b$ for the events of a process, then $C(a) < C(b)$
- If event a is the sending of a message, and b is the receipt of this message, then $C(a) < C(b)$.

If there is a global clock, then the timestamp can be made. In what follows we deal with what happens if there is no global clock.

Lamport timestamp: Every process P_i keeps track of a counter C_i as follows:

- Every event of P_i increases C_i by one.
- On the sent message m we write the timestamp: $ts(m) = C_i$.

- If the message m arrives at process P_j , then the new value of the counter will be $C_j = \max\{C_j, ts(m)\} + 1$
- Of the coinciding timestamps of P_i and P_j let us regard the one in P_i as first, if $i < j$.

Totally ordered multicast: Process P_i sends every operation in a timestamped message. P_i at the same time puts the sent message into its own $queue_i$ priority queue. Process P_j puts the incoming messages into its $queue_j$ priority queue with priority corresponding to the timestamp. It notifies every process about the arrival of the message. P_j hands over the message msg_i for processing if:

- msg_i is found at the front of $queue_j$, that is, its timestamp is the smallest
- in the $queue_j$ queue, for every process $P_k, k \neq i$, at least one message of it is found whose timestamp is later than that of msg_i

Timestamp vector (vector clock):

- P_i now also keeps track of the time of all processes in an array $VC_i[1..n]$, where $VC_i[j]$ is the number of events occurring in P_j that P_i knows about.
- During the sending of message m , P_i increases the value of $VC_i[i]$ by one and writes the whole VC_i timestamp vector onto the message.
- When message m arrives at P_j , which has the timestamp $ts(m)$ on it, then
 1. $VC_j[k] := \max\{VC_j[k], ts_m[k]\}$
 2. $VC_j[j]$ increases by one

5.2 Mutual exclusion

Several processes want to access a given resource at once. This we can allow only to one of them at a time, otherwise the resource can get into an incorrect state.

5.2.1 Mutual exclusion using a central server

A central server is the coordinator, it regulates the access to the resource. It has a waiting queue. If the resource is free, then if a request arrives for it, the server grants the access and makes it occupied. After this, if someone else wants to access the resource, then they get into the waiting queue. After the first client releases the resource, it goes to the one who is at the front of the queue. If the queue has emptied and the last client has also released the resource, it becomes free again.

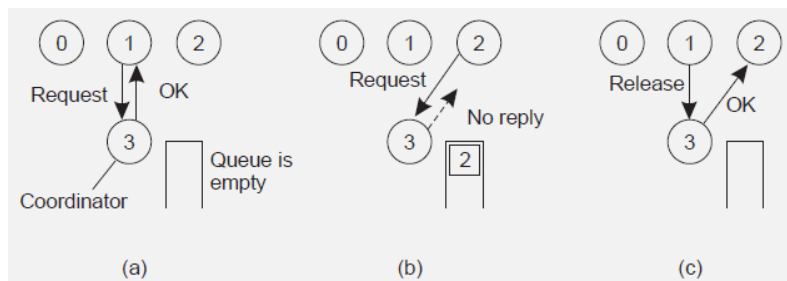


Figure 9: Example of centralized mutual exclusion.

5.2.2 Decentralized mutual exclusion

Suppose that the resource is n -fold replicated, and to every replica belongs a coordinator handling it. The access is decided by majority voting: at least m coordinators are needed, where $m > \frac{n}{2}$. We assume that after a possible crash the coordinator recovers, but forgets the granted permissions.

5.2.3 Distributed mutual exclusion

The resource is replicated. When the client wants to access the resource, it sends a request to the coordinator provided with a timestamp. It gets a response (access permission) if:

- The coordinator does not need the resource, or
- the coordinator also needs the resource, but its timestamp is smaller.
- Otherwise the coordinator temporarily does not respond.

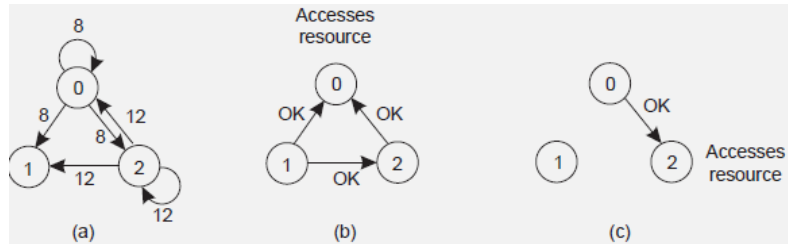


Figure 10: Example of distributed mutual exclusion.

5.2.4 Mutual exclusion with a token ring

We organize the processes into a logical ring. We send a token around. Whichever process holds the token can access the resource.

5.3 Leader election

Many algorithms need to designate a process that coordinates the further steps.

5.3.1 Bully algorithm

We give the processes a sequence number, of which we want to choose the largest-sequence-number one as leader.

The steps of the bully algorithm:

1. Initiation of the leader election. Any process can initiate it. To every process about which it does not know that its sequence number would be smaller than its own, it sends a message.
2. If the larger-sequence-number process gets a message from a smaller-sequence-number one, then it sends back a message with which it takes the smaller-sequence-number one out of the election.
3. Whichever process does not get a disabling message within a certain time, that one will be the leader. About this it notifies the other processes with a message each.

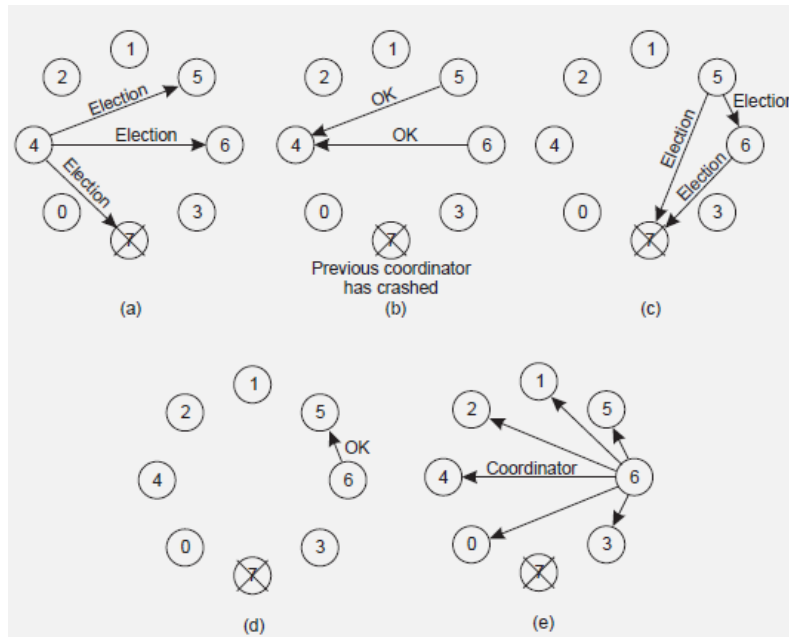


Figure 11: Example of the operation of the bully algorithm.

5.3.2 Leader election in a ring

We have a logical ring, the processes have sequence numbers. We want to choose the largest-sequence-number process as leader. Any process can initiate a leader election: it starts a message around the ring, onto which everyone writes their sequence number. If a process has crashed, it is left out of the message sending. When the message gets back to the initiator, the sequence number of every active process appears on it. Of these the largest-sequence-number one will be the leader. This is made known to everyone by sending another message around.

If several processes initiate an election at once, that is not a problem, the same result is obtained. If the messages were lost, then the election can be started over.

5.3.3 Superpeer election

We want to choose the superpeers so that the following hold for them:

- The other vertices reach them with low latency.
- They are evenly distributed over the network.
- We choose a given fraction of the vertices as superpeer.
- A superpeer serves a limited number of peers.

Realization in case of DHT: If we use m -bit identifiers, and S superpeers are needed, then let us reserve the $k = \lceil \log_2 S \rceil$ top bits for the superpeers. Thus in case of N vertices there will be about $2^{k-m}N$ superpeers.

The superpeer belonging to key p will be the one responsible for the key p AND $\underbrace{11\dots11}_k \underbrace{00\dots00}_{m-k}$.

6 Consistency

Conflicting operations: To keep the replicas consistent, it has to be ensured that the operations that can get into conflict with each other run in the same order on every replica. Read-write and write-write conflicts can occur.

Consistency model: The consistency model prescribes in which ways the processes can use the database. If the conditions hold, we regard the data store as valid.

Degree of consistency: The consistency can be violated in several ways: the replicas' numeric value, freshness, the number of not-yet-performed update operations can differ.

Conit: The data unit to which a common condition system relates is the conit (consistency unit).

6.1 Sequential consistency

We base the conditions not on numeric values, but on the fact of writes/reads. Notations:

- $W(x)$: the process wrote variable x
- $R(x)$: the process read variable x

In the case of sequential consistency we expect that the result of the execution be as if all operations of all processes had happened in a determined order, preserving the order of the own operations of any given process.

P1: $W(x)a$	P1: $W(x)a$
P2: $W(x)b$	P2: $W(x)b$
P3: $R(x)b$ $R(x)a$	P3: $R(x)b$ $R(x)a$
P4: $R(x)b$ $R(x)a$	P4: $R(x)a$ $R(x)b$
(a)	(b)

Figure 12: Example: (a) satisfies, (b) does not satisfy the requirements of sequential consistency.

6.2 Causal consistency

The potentially causally related operations have to be seen by every process in the same order. The concurrent writes can be seen by the different processes in different orders.

P1: $W(x)a$	P1: $W(x)a$
P2: $R(x)a$ $W(x)b$	P2: $W(x)b$
P3: $R(x)b$ $R(x)a$	P3: $R(x)b$ $R(x)a$
P4: $R(x)a$ $R(x)b$	P4: $R(x)a$ $R(x)b$
(a)	(b)

Figure 13: Example: (b) satisfies, (a) does not satisfy the requirements of causal consistency.

6.3 Client-centric consistency

We now put in the foreground how the data stored on the servers appears to a given client. The client moves: it connects to different servers, and performs write/read operations.

Connecting to server B after server A , different problems can arise:

- The updates uploaded to A may not yet have reached B .
- On B there may be newer data than on A .

- The updates uploaded to B can conflict with those uploaded to A .

The goal is that the client see the data that it handled on server A in the same state on B too. Then the database appears consistent to the client.

6.3.1 Monotonic reads

If the client once read out a value from x , every reading following this gives this, or a fresher value.

For example, in the case of an email client, every previously downloaded letter of ours has to be present on the new server too.

6.3.2 Monotonic writes

The client can write x if the client's earlier writes to x have already finished.

For example, in version control, every earlier version has to be present on the server if we want to upload a new version.

6.3.3 Read your writes

If the client reads x , it gets the result of its own most recent write, or a fresher one.

For example, the client edits their homepage, then looks at the result. Instead of an older version coming up from the browser's cache, they want to see the freshest one.

6.3.4 Writes follow reads

If the client read out a value from x , every update operation issued after this modifies an at least this fresh value of x .

For example, on a forum the client can only respond to a post that they have already seen.

6.4 Replication of content

Different kinds of processes can store the copies:

- Permanent copy: origin server
- Server-initiated copy: placing a replica on a server when it needs the data
- Client-initiated copy: client-side cache

6.4.1 Spreading of updates

Changed content can be passed in several different ways in a client-server architecture:

- Spreading exclusively the notification/invalidation about the update.
- Passive replication: transferring data from one copy to another
- Active replication: transferring the update operation

The update can be initiated by the server (push-based update), in which case the server sends the update to the client without the client's request, or it can be initiated by the client, who requests the update from the server (pull-based update).

Lease: The server makes a promise to the client that it sends the update over as long as the lease is active.